DeepGuard API

Video Deepfake Detection API

Frontend Integration Guide for Developers

Version 3.0.0

June 2026

Base URL	`https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io`
API Type	`REST (JSON)`
Authentication	`None (CORS: allow_origins=["*"])`
Format	`multipart/form-data (upload) / JSON (response)`
Report Format	`PDF (application/pdf)`

Table of Contents

1. API Overview & Endpoints

DeepGuard is a production-ready deepfake detection API that analyzes videos using two independent biological signals: **visual appearance** (ResNet-18 + ViT-B/16) and **physiological pulse** (rPPG via CHROM/POS algorithms). It returns a verdict with confidence scores, per-model breakdown, and a downloadable PDF forensic report.

Method	Endpoint	Description
GET	`/`	Health check. Returns {"status":"ready"}
POST	`/detect`	Upload video → JSON verdict + report_id
GET	`/report/{report_id}`	Download PDF forensic report

2. POST /detect — Upload & Analyze

Endpoint: `POST /detect`

Content-Type: `multipart/form-data`

Body Parameter: `file` (the video file)

Supported Formats

Format	Extension	Notes
MP4	.mp4	H.264 encoded, preferred
AVI	.avi	May be larger file size
MOV	.mov	QuickTime container
WebM	.webm	VP8/VP9 codec
FLV	.flv	Flash video

cURL Example

```
curl -X POST \
  -F "file=@video.mp4" \
  https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/detect
```

JavaScript (fetch) Example

```
async function checkDeepfake(videoFile) {
  const formData = new FormData();
  formData.append('file', videoFile);

  const res = await fetch(
    'https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/detect',
    { method: 'POST', body: formData }
  );

  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(err.detail || 'Detection failed');
  }

  return await res.json();
}
```

Python Example

```
import requests

url = "https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/detect"

with open("video.mp4", "rb") as f:
    files = {"file": ("video.mp4", f, "video/mp4")}
    res = requests.post(url, files=files, timeout=180)

if res.status_code == 200:
    data = res.json()
    print(f"Decision: {data['decision']}")
```

```
    print(f"Reason: {data['reason']}")
    print(f"Report ID: {data['report_id']}")
else:
    print(f"Error {res.status_code}: {res.text}")
```

3. GET /report/{report_id} — Download PDF

After a successful detection, the API returns a `report_id` (8-character hex string). Use it to download a professional PDF forensic report.

Endpoint: `GET /report/{report_id}`

Response: `application/pdf` (binary download)

cURL Example

```
curl -o forensic_report.pdf \
  https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/report/ee96c81b
```

JavaScript (Download Trigger) Example

```
async function downloadReport(reportId) {
  const res = await fetch(
    `https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/report/${reportId}`
  );

  if (!res.ok) {
    throw new Error('Report not found or expired');
  }

  const blob = await res.blob();
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `deepfake_report_${reportId}.pdf`;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}
```

PDF Report Contents

• Decision badge (color-coded: green=REAL, red=FAKE, yellow=UNCERTAIN, gray=NO_FACE)
• Video file info (filename, duration, frame count)
• Visual stream metrics table (ResNet-18 + ViT-B/16 scores) with reference baselines
• rPPG stream metrics table (BPM, HRV, SNR, kurtosis) with color-coded health assessments
• rPPG waveform chart with detected pulse peaks (matplotlib-generated chart embedded as PNG)
• Face crop thumbnails (up to 3 sample frames)
• Fusion features table with color-coded highlights
• Pipeline metadata footer (model versions, inference time)
• Metrics Reference Glossary appendix (plain-English explanations of every metric)

> ***Important:*** *Reports are cached in memory for* ***10 minutes*** *after detection. If the user downloads later, they must re-upload the video. The report_id expires after 10 minutes and returns 404.*

4. POST /detect — Complete Response Fields Reference

This is the full JSON response structure. Every field is explained below.


```
{
  "video": "echo_mimic.mp4",
  "decision": "FAKE",
  "confidence": 1.0,
  "prob_real": 0.0,
  "visual_score": 0.999,
  "temporal_std": 0.0006,
  "per_model_scores": {
    "resnet18": 0.998,
    "vit_b16": 1.0
  },
  "rppg": {
    "bpm": 0.0,
    "bpm_var": 0.0,
    "hrv_rmssd": 0.0,
    "hrv_sdnn": 0.0,
    "snr": 0.0,
    "skew": 0.0,
    "kurt": 0.0
  },
  "features": {
    "visual_score": 1.0,
    "temporal_std": 0.0,
    "bpm": 0.0,
    "bpm_var": 0.0,
    "hrv_rmssd": 0.0,
    "hrv_sdnn": 0.0,
    "snr": 0.0,
    "skew": 0.0,
    "kurt": 0.0
  },
  "reason": "Frozen face — temporal variance near zero",
  "report_id": "ee96c81b"
}
```


Field	Type	Range	Description
decision	string	REAL / FAKE / UNCERTAIN / NO_FACE	Primary verdict. Use this for your UI badge/display.
confidence	float	0.0 – 1.0	How confident the system is in its decision.
prob_real	float	0.0 – 1.0	Raw probability the video is authentic human speech.
reason	string	—	Human-readable explanation of why the decision was reached. Display this in your UI.
report_id	string	8-char hex	Use with GET /report/{id} to download PDF. Expires after 10 minutes.
visual_score	float	0.0 – 1.0	Visual model score. 1.0 = definitely a deepfake. 0.0 = definitely real.
temporal_std	float	0.0 – 1.0+	Frame-to-frame pixel variance. Near 0 = frozen face (instant FAKE).
per_model_scores.resnet18	float	0.0 – 1.0	ResNet-18 individual score.
per_model_scores.vit_b16	float	0.0 – 1.0	ViT-B/16 individual score.

rppg.bpm	float	0.0 – 200.0	Estimated heart rate from facial pulse. 0 = no pulse detected.
rppg.hrv_rmssd	float	0.0 – 500.0	Heart rate variability (RMSSD). Higher = more natural variation.
rppg.snr	float	-10.0 – 10.0+	rPPG signal-to-noise ratio. Positive = good signal.
rppg.kurt	float	0.0 – 50.0+	Pulse waveform kurtosis. Normal ~3. Extreme values = abnormal morphology.
features.*	float	Varies	All 9 features normalized for the fusion engine. Not typically needed for frontend display.

5. Decision Rules & Their Meanings

The decision engine follows a strict priority order. Understanding these rules helps you interpret the `reason` field in the response.

Priority	Condition	Decision	Reason String
1	No face detected in any frame	NO_FACE	No face detected in any frame
2	temporal_std < 0.001	FAKE	Frozen face — temporal variance near zero
3	rPPG kurtosis > 10 or < 0.5 + inflated HRV	FAKE	Abnormal rPPG morphology — spikey or flat pulse
4	SNR < -2.0 + inflated HRV	FAKE	Low rPPG signal quality — noise masquerading as pulse
5	No rPPG data + prob_real < 0.85	UNCERTAIN	No physiological confirmation — insufficient evidence
6	Valid pulse (BPM 50–120, SNR > 0) + prob_real ≥ 0.85	REAL	Standard weighted fusion + valid pulse override
7	Default weighted fusion	REAL/FAKE/UNCERTAIN	Standard weighted fusion

What to display for each decision:

Decision	UI Badge Color	Recommended Icon	Example Message
`REAL`	#22c55e (Green)	✓	Likely authentic human video
`FAKE`	#ef4444 (Red)	✗	AI-generated or manipulated video detected
`UNCERTAIN`	#f59e0b (Amber)	?	Insufficient evidence — manual review needed
`NO_FACE`	#6b7280 (Gray)	—	No face detected in video

6. Frontend Integration Examples

React Component — Full Example

```
import React, { useState } from 'react';

const API_BASE = 'https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io';

const DECISION_COLORS = {
  REAL: '#22c55e',
  FAKE: '#ef4444',
  UNCERTAIN: '#f59e0b',
  NO_FACE: '#6b7280'
};

const DECISION_LABELS = {
  REAL: 'Real',
  FAKE: 'Fake',
  UNCERTAIN: 'Uncertain',
  NO_FACE: 'No Face'
};

function DeepfakeChecker() {
  const [file, setFile] = useState(null);
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState(null);
  const [error, setError] = useState(null);

  const handleUpload = async () => {
    if (!file) return;
    setLoading(true);
    setError(null);
    setResult(null);

    const formData = new FormData();
    formData.append('file', file);

    try {
      const res = await fetch(`${API_BASE}/detect`, {
        method: 'POST',
        body: formData
      });

      if (!res.ok) {
        const err = await res.json().catch(() => ({}));
        throw new Error(err.detail || 'Detection failed');
      }

      const data = await res.json();
      setResult(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const downloadReport = async () => {
    if (!result?.report_id) return;
    const res = await fetch(`${API_BASE}/report/${result.report_id}`);
    const blob = await res.blob();
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `deepfake_report_${result.report_id}.pdf`;
    a.click();
    URL.revokeObjectURL(url);
  };
```

```
return (
  <div style={{ maxWidth: 600, margin: '0 auto', padding: 24 }}>
    <h1>DeepGuard Detector</h1>

    <input
      type="file"
      accept="video/*"
      onChange={e => setFile(e.target.files[0])}
    />

    <button onClick={handleUpload} disabled={!file || loading}>
      {loading ? 'Analyzing...' : 'Check Video'}
    </button>

    {error && (
      <div style={{ color: '#ef4444', marginTop: 16 }}>
        {error}
      </div>
    )}

    {result && (
      <div style={{
        marginTop: 24,
        padding: 16,
        borderRadius: 8,
        borderLeft: `4px solid ${DECISION_COLORS[result.decision]}`
      }}>
        <h2 style={{ fontSize: 24, margin: 0 }}>
          {DECISION_LABELS[result.decision]}
        </h2>

        <p style={{ color: '#718096', marginTop: 4 }}>
          {result.reason}
        </p>

        <div style={{
          display: 'grid',
          gridTemplateColumns: '1fr 1fr',
          gap: 8,
          marginTop: 16,
          fontSize: 14
        }}>
          <div>
            Confidence: <strong>{(result.confidence * 100).toFixed(0)}%</strong>
          </div>
          <div>
            Visual Score: <strong>{(result.visual_score * 100).toFixed(0)}%</strong>
          </div>
          <div>
            Heart Rate: <strong>{result.rppg.bpm?.toFixed(0) || 'N/A'} BPM</strong>
          </div>
          <div>
            Temporal Variance: <strong>{result.temporal_std?.toFixed(4)}</strong>
          </div>
        </div>

        <div style={{ marginTop: 8, fontSize: 13, color: '#718096' }}>
          ResNet-18: {(result.per_model_scores.resnet18 * 100).toFixed(0)}% |
          ViT-B/16: {(result.per_model_scores.vit_b16 * 100).toFixed(0)}%
        </div>

        <button
          onClick={downloadReport}
          style={{
            marginTop: 16,
            padding: '8px 16px',
            background: '#2b6cb0',
            color: 'white',
            border: 'none',
            borderRadius: 4,
            cursor: 'pointer'
```

```
            }}
          >
            Download PDF Report
          </button>
        </div>
      )}
    </div>
  );
}
```

Vue.js Example (Composition API)

```
<template>
  <div>
    <input type="file" accept="video/*" @change="onFileChange" />
    <button @click="upload" :disabled="!file || loading">
      {{ loading ? 'Analyzing...' : 'Check Video' }}
    </button>

    <div v-if="error" class="error">{{ error }}</div>

    <div v-if="result" class="result-card">
      <h2 :style="{ color: colors[result.decision] }">
        {{ labels[result.decision] }}
      </h2>
      <p class="reason">{{ result.reason }}</p>
      <p>Confidence: {{ (result.confidence * 100).toFixed(0) }}%</p>
      <p>BPM: {{ result.rppg.bpm?.toFixed(0) || 'N/A' }}</p>
      <button @click="downloadReport">Download PDF Report</button>
    </div>
  </div>
</template>

<script setup>
import { ref } from 'vue';

const API_BASE = 'https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io';
const file = ref(null);
const loading = ref(false);
const result = ref(null);
const error = ref(null);
const colors = { REAL: '#22c55e', FAKE: '#ef4444', UNCERTAIN: '#f59e0b', NO_FACE: '#6b7280' };
const labels = { REAL: 'Real', FAKE: 'Fake', UNCERTAIN: 'Uncertain', NO_FACE: 'No Face' };

const onFileChange = (e) => { file.value = e.target.files[0]; };

const upload = async () => {
  if (!file.value) return;
  loading.value = true;
  error.value = null;
  result.value = null;

  try {
    const fd = new FormData();
    fd.append('file', file.value);
    const res = await fetch(`${API_BASE}/detect`, { method: 'POST', body: fd });
    if (!res.ok) throw new Error((await res.json()).detail);
    result.value = await res.json();
  } catch (e) {
    error.value = e.message;
  } finally {
    loading.value = false;
  }
};

const downloadReport = async () => {
  if (!result.value?.report_id) return;
  const res = await fetch(`${API_BASE}/report/${result.value.report_id}`);
  const blob = await res.blob();
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `deepfake_report_${result.value.report_id}.pdf`;
  a.click();
  URL.revokeObjectURL(url);
};
</script>
```

7. Error Handling Guide

Every error returns a JSON body with a `detail` field. Your frontend should always check `res.ok` before parsing the response.

Scenario	Status	Response	Frontend Action
No file uploaded	400	`{"detail": "No file provided."}`	Show 'Please select a video file'
Empty filename	400	`{"detail": "No file provided."}`	Show 'Please select a video file'
Cannot read video frames	400	`{"detail": "Could not read video frames. Unsupported video format or corrupted file"}`	Show 'Unsupported video format or corrupted file'
No face detected	200	`{"decision": "NO_FACE", ...}`	Show gray badge with 'No face detected'
Report expired	404	`{"detail": "Report not found or expired."}`	Show 'Report expired. Please re-upload video.'
Server error	500	`Internal Server Error`	Show 'Server error. Try again.' + retry button
Timeout (cold start)	502/504	`Gateway Timeout`	Show 'First load takes ~60s. Please retry.'

Recommended Error Handling Pattern

```
async function safeDetect(videoFile) {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), 180000);

  try {
    const fd = new FormData();
    fd.append('file', videoFile);

    const res = await fetch(
      'https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/detect',
      { method: 'POST', body: fd, signal: controller.signal }
    );

    if (!res.ok) {
      const body = await res.json().catch(() => ({}));
      const msg = body.detail || `Server error (${res.status})`;
      throw new Error(msg);
    }

    return await res.json();
  } catch (err) {
    if (err.name === 'AbortError') {
      throw new Error('Request timed out. The video may be too large.');
    }
    throw err;
  } finally {
    clearTimeout(timeout);
  }
}
```

8. Performance & Limitations

Characteristic	Value	Notes
Cold start time	~60 seconds	First request after idle scales up replica
Inference time (30s video)	30–90 seconds	CPU-only. Depends on video length.
Max video duration	~30 seconds	Configured: 90 frames at 3 fps
Frame sample rate	3 fps	Nyquist limit: 90 BPM for rPPG
Max file size	~100 MB	Container Apps default limit
Concurrent requests	Up to 3	Max replicas configured
Report cache TTL	10 minutes	After that, re-upload required
Supported formats	MP4, AVI, MOV, WebM, FLV	H.264 recommended

Recommendations for frontend: Show a progress indicator/loading spinner during detection. Set a 180-second timeout on your fetch call. On first load of the day, consider a pre-warm ping (GET /) to start the container before the real request.

9. CORS & Security

The API is configured with wide-open CORS for development flexibility:

```
// Current CORS configuration:
allow_origins=["*"]
allow_methods=["*"]
allow_headers=["*"]
expose_headers=["Content-Disposition"]
```

This means any web origin can call the API from the browser. However, credentialed requests (`credentials: 'include'`, cookies, HTTP auth) will fail with CORS errors. If your frontend sends credentials, we need to switch to a specific origin list.

If your dashboard sends credentials:

Send me the list of origins that will make requests (e.g., `['https://your-app.com']`), and I will update the server configuration.

To test CORS in the browser:

```
// Open browser console and run:
fetch('https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io/', {
  method: 'GET'
}).then(r => console.log('CORS OK:', r.status))
  .catch(e => console.error('CORS ERROR:', e));
```

If you see a response, CORS is working. If you get a CORS error in the console, inspect the browser's network tab for the exact error message.

10. Quick-Start Cheat Sheet

Copy-paste ready:

```
// === 1. Upload video & get result ===
const API = 'https://deepfake-api.reddune-ee354d90.francecentral.azurecontainerapps.io';

async function detect(videoFile) {
  const fd = new FormData();
  fd.append('file', videoFile);
  const res = await fetch(`${API}/detect`, { method: 'POST', body: fd });
  return await res.json();
}

// === 2. Display result ===
function renderResult(data) {
  return {
    badge: data.decision,           // "REAL" | "FAKE" | "UNCERTAIN" | "NO_FACE"
    color: {                        // Badge colors
      REAL: '#22c55e',
      FAKE: '#ef4444',
      UNCERTAIN: '#f59e0b',
      NO_FACE: '#6b7280'
    }[data.decision],
    label: data.reason,             // Human-readable explanation
    confidence: data.confidence,    // 0-1
    reportId: data.report_id        // For PDF download
  };
}

// === 3. Download PDF report ===
async function download(reportId) {
  const res = await fetch(`${API}/report/${reportId}`);
  const blob = await res.blob();
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `deepfake_report_${reportId}.pdf`;
  a.click();
  URL.revokeObjectURL(url);
}

// === 4. Check that the API is alive ===
async function health() {
  const res = await fetch(API);
  const data = await res.json();
  return data.status === 'ready';
}
```